ECE 264 - Object-Oriented Software Development

Chapter 6

Dictionaries and Sets

Core Objectives

- Use dictionaries to represent unordered collections of key–value pairs.
- Use sets to represent unordered collections of unique values.
- Create, initialize and refer to elements of dictionaries and sets.
- Iterate through a dictionary’s keys, values and key–value pairs.
- Add, remove and update a dictionary’s key–value pairs.
- Operators and methods

Introduction

- A **dictionary** is an *unordered* collection which stores **key–value pairs** that map immutable keys to values, just as a conventional dictionary maps words to definitions.
- A **set** is an unordered collection of unique immutable elements.

Dictionaries

- A dictionary *associates* keys with values.
- Each key *maps* to a specific value.
- Keys must be *immutable* and *unique*.
- Multiple keys can have the same value.
- Sample dictionary keys and values:

Keys	Key type	Values	Value type
Country names	`str`	Internet country codes	`str`
Decimal numbers	`int`	Roman numerals	`str`
States	`str`	Agricultural products	list of `str`
Hospital patients	`str`	Vital signs	tuple of `int` s and `float` s
Baseball players	`str`	Batting averages	`float`
Metric measurements	`str`	Abbreviations	`str`
Inventory codes	`str`	Quantity in stock	`int`

Creating a Dictionary

- Create a dictionary by enclosing in curly braces, {}, a comma-separated list of key–value pairs, each of the form *key*: *value*.
- Create an empty dictionary with {}.
- Dictionaries are *unordered* collections.
- Do *not* write code that depends on the order of the key–value pairs.
- Can use a dictionary as a condition to determine if it's empty -- non-empty is `True` and empty is `False`

Iterating through a Dictionary

- Dictionary method **items** returns each key–value pair as a tuple

```
days_per_month = {'January': 31, 'February': 28, 'March': 31}
```

```
for month, days in days_per_month.items():
    print(f'{month} has {days} days')
```

```
January has 31 days
February has 28 days
March has 31 days
```

Basic Dictionary Operations

- Accessing the Value Associated with a Key
- Updating the Value of an Existing Key–Value Pair
- Adding a New Key–Value Pair
  - String keys are case sensitive.
  - Assigning to a nonexistent key inserts a new key–value pair.
- Removing a Key–Value Pair
- Method `pop` returns the value for the removed key.
- Method `get` returns its argument's corresponding value or None if the key is not found.
- Operators `in` and `not in` can determine whether a dictionary contains a specified key

Basic Dictionary Operations

- Dictionary Methods `keys` and `values`
- Methods `items`, `keys` and `values` each return a **view** of a dictionary's data.
- When you iterate over a view, it "sees" the dictionary's current contents – it does not have its own copy of the data.
- Converting Dictionary Keys, Values and Key–Value Pairs to Lists
- Processing Keys in Sorted Order
- Comparison - `==` is `True` if both dictionaries have the same key–value pairs, regardless of the order in which those key–value pairs were added to each dictionary.
- Sorting – To process keys in sorted order, you can use built-in function `sorted`.

Practice – Grade Book

```python
# fig06_01.py
"""Using a dictionary to represent an instructor's grade book."""
grade_book = {
    'Susan': [92, 85, 100],
    'Eduardo': [83, 95, 79],
    'Azizi': [91, 89, 82],
    'Pantipa': [97, 91, 92]
}

all_grades_total = 0
all_grades_count = 0

for name, grades in grade_book.items():
    total = sum(grades)
    print(f'Average for {name} is {total/len(grades):.2f}')
    all_grades_total += total
    all_grades_count += len(grades)

print(f"Class's average is: {all_grades_total / all_grades_count:.2f}")
```

```
Average for Susan is 92.33
Average for Eduardo is 85.67
Average for Azizi is 87.33
Average for Pantipa is 93.33
Class's average is: 89.67
```

Practice – Word Counts

- Builds a dictionary to count the number of occurrences of each word in a string.

```
# fig06_02.py
"""Tokenizing a string and counting unique words."""

text = ('this is sample text with several words '
        'this is more sample text with some different words')

word_counts = {}

# count occurrences of each unique word
for word in text.split():
    if word in word_counts:
        word_counts[word] += 1  # update existing key-value
pair
    else:
        word_counts[word] = 1  # insert new key-value pair

print(f'{"WORD":<12}COUNT')

for word, count in sorted(word_counts.items()):
    print(f'{word:<12}{count}')

print('\nNumber of unique words:', len(word_counts))
```

`update` Method

- Can insert and update key–value pairs.
- Method update also can receive an iterable object containing key–value pairs, such as a list of two-element tuples.

```
country_codes = {}
```

```
[ ]:
```

```
country_codes.update({'South Africa': 'za'})
```

Dictionary Comprehensions

- Convenient notation for quickly generating dictionaries, often by **mapping** one dictionary to another.
- The expression to the left of the for clause specifies a **key–value pair of the form *key*: *value***.
- In a dictionary with ***unique* values**, you can **reverse** the key–value pair mappings.
- A dictionary comprehension also can map a dictionary's values to new values.

Sets

- A set is an unordered collection of unique values.
- May contain only immutable objects, like strings, ints, floats and tuples that contain only immutable elements.
- Sets do not support indexing and slicing.
- Creating a set with curly braces. Duplicates are ignored, making sets great for duplicate elimination.
- `in` and `not in`
- There's no significance to the iteration order.
- Creating a Set with the Built-In set Function
  - To create an empty set, must use the **set()**, because **{} represents an empty dictionary**.
  - Python displays an empty set as set() to avoid confusion with an empty dictionary ({}).

Operations

- Comparison
  - ==
  - !=
  - <
    - tests whether the set to its left is a proper subset of the one to its right—all the elements in the left operand are in the right operand, and the sets are not equal.
  - <=
    - tests whether the set to its left is an improper subset of the one to its right—that is, all the elements in the left operand are in the right operand, and the sets might be equal.
  - You may also check for an improper subset with the set method issubset:
  - you may also check for a proper superset with > and improper supersets with >= or set method issuperset.

Mathematical Set Operations

- Union
  - The union of two sets is a set consisting of all the unique elements from both sets.
  - The union operator requires two sets, but method union may receive any iterable as its argument (this is true for subsequent methods in this section as well).
- Intersection
  - The intersection of two sets is a set consisting of all the unique elements that the two sets have in common.
- Difference
  - The difference between two sets is a set consisting of the elements in the left operand that are not in the right operand.
- Symmetric Difference
  - The **symmetric difference** between two sets is a set consisting of the elements of both sets that are not in common with one another.
- Disjoint
  - Two sets are **disjoint** if they do not have any common elements.

Mutable Set Operators and Methods

- Mutable Set Methods
  - Intersection augmented assignment &=
  - Difference augmented assignment -=
  - Symmetric difference augmented assignment ^=
- Methods with iterable arguments:
  - `intersection_update`
  - `difference_update`
  - `symmetric_difference_update`
- `add`, `remove`, `pop`, `clear`